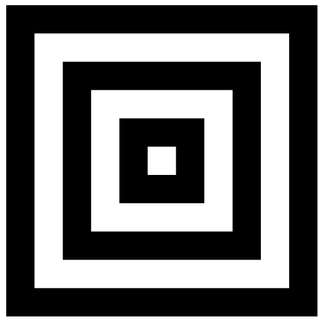
THE UNIVERSITY OF MELBOURNE

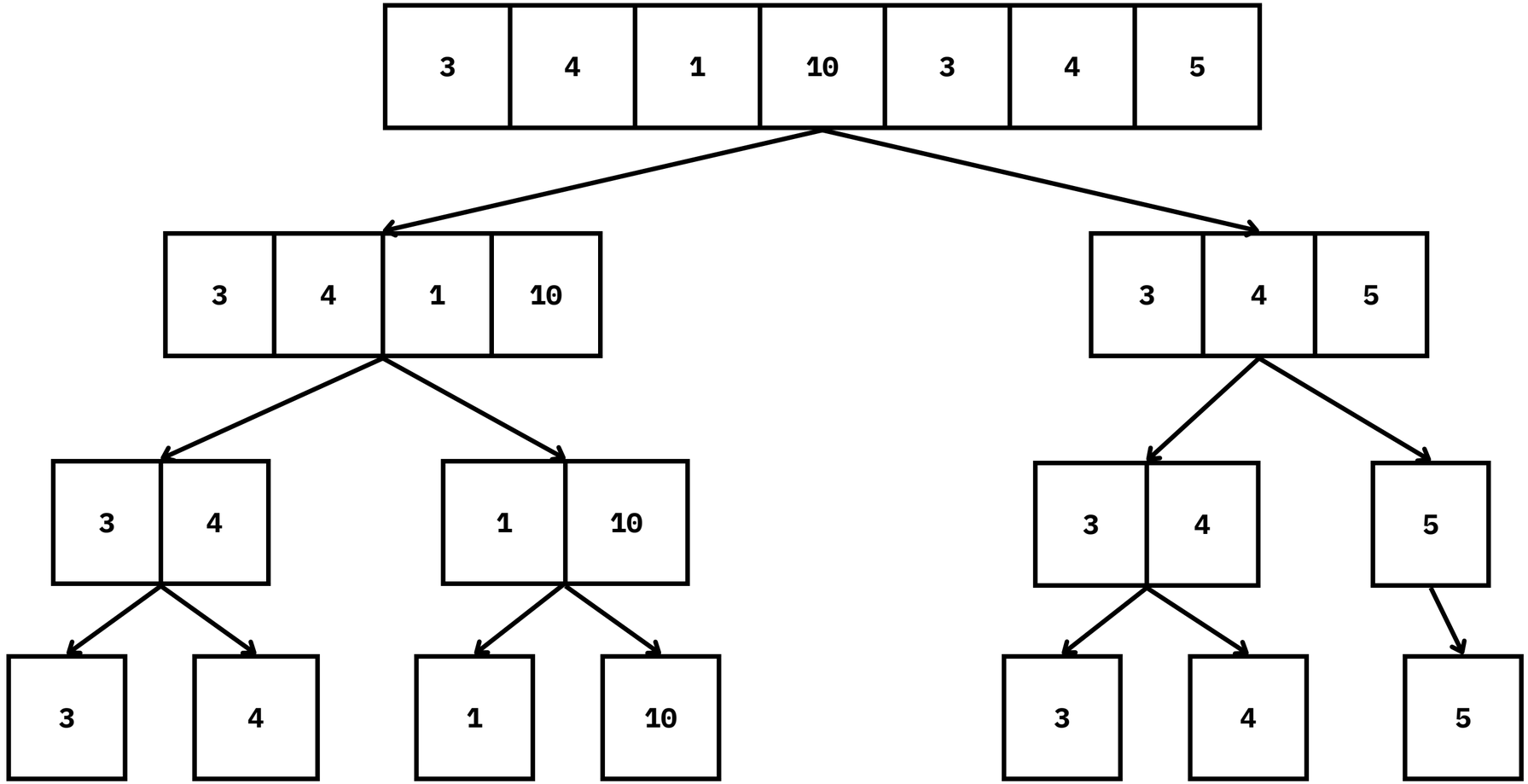
COMP10001 - Sem 2 2024 - Week 11

Foundations of Computing

Daksh Agrawal



Recursion


3 4 1 10 3 4 5
3 4 1 10
3 4 5
3 4
1 10
3 4
5
3
4
1
10
3
4
5


JETPACK JOYRIDE
DOODLE JUMP
HILL CLIMB RACING
GEOMETRY DASH
TALKING TOM
FRUIT NINJA
WHERE'S MY WATER
TEMPLE RUN
SUBWAY SURFERS
PAPER TOSS
ANGRY BIRDS
CLASH OF CLANS
CANDY CRUSH SAGA
ASPHALT 6
SHADOW GUN
FLAPPY BIRD

What makes a function recursive?

```
1    def max_in_list(l):
2        """ This function returns the maximum value in the list l. """
3        if len(l) == 1:
4            return l[0]
5        else:
6            return max(l[0], max_in_list(l[1:]))
```

What are the 2 parts of a recursive function?

```
1    def max_in_list(l):
2        """ This function returns the maximum value in the list l. """
3        if len(l) == 1:
4            return l[0]
5        else:
6            return max(l[0], max_in_list(l[1:]))
```

base case / part

recursive case / part

When is recursion useful?

Caution

```
9   def mystery(x):
10      if len(x) == 1:
11          return x[0]
12      else:
13          y = mystery(x[1:])
14          if x[0] > y:
15              return x[0]
16          else:
17              return y
18
```

base case

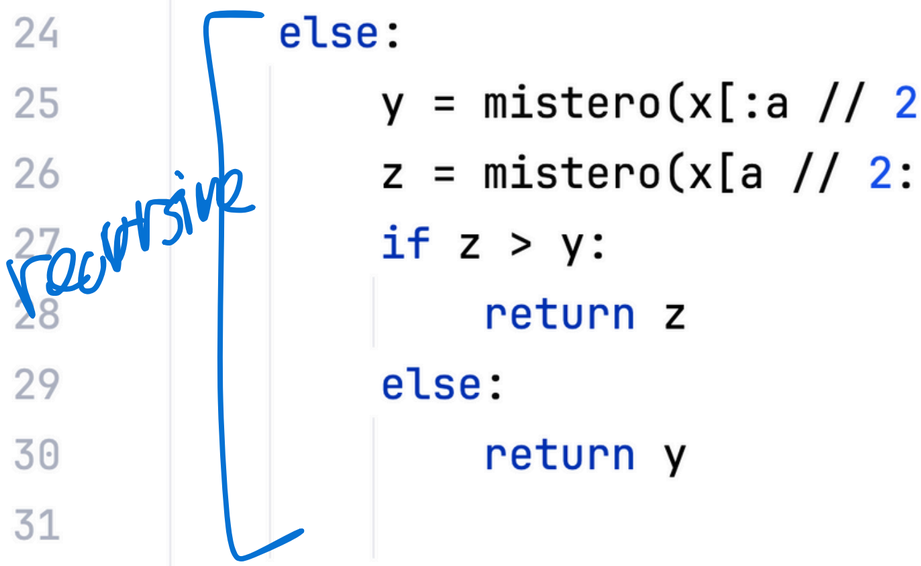
recursive case

Base Case?

Recursive Case?

What does the function do?

```
20  def mistero(x):
21      a = len(x)
22      if a == 1:
23          return x[0]
24      else:
25          y = mistero(x[:a // 2])
26          z = mistero(x[a // 2:])
27          if z > y:
28              return z
29          else:
30              return y
31
```



Base Case?

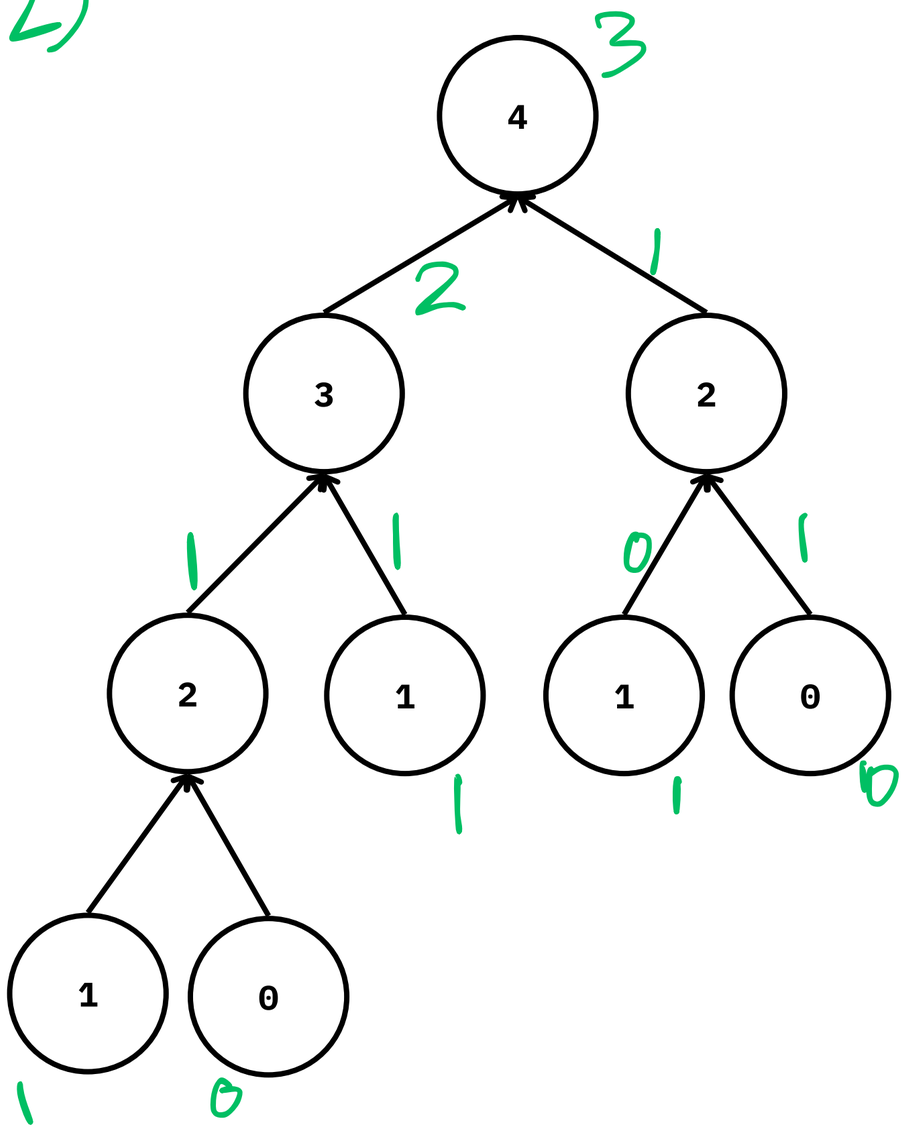
Recursive Case?

What does the function do?

$$fib(n) = fib(n-1) + fib(n-2)$$

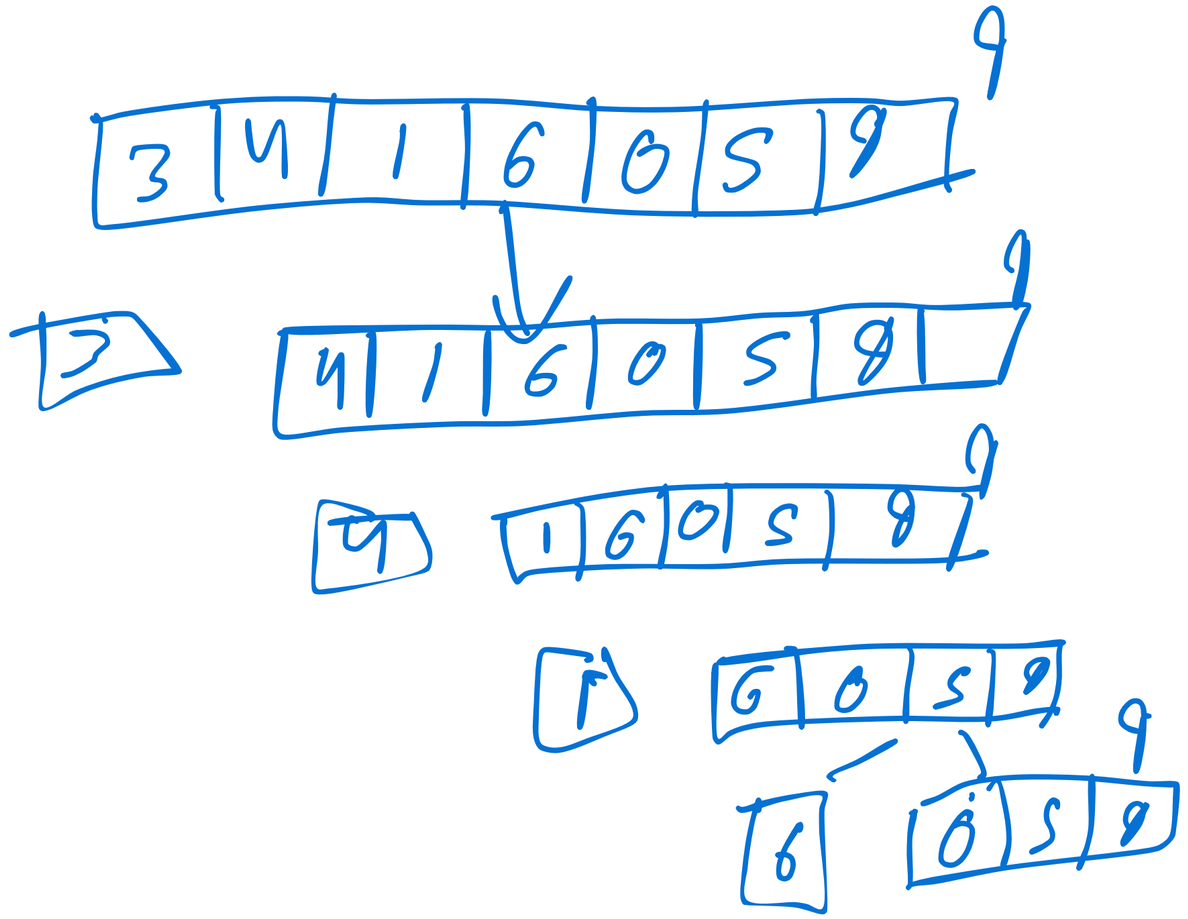
```
33  def fib(n):
34      if n in [0, 1]:
35          return n
36      return fib(n - 1) + fib(n - 2)
```

base



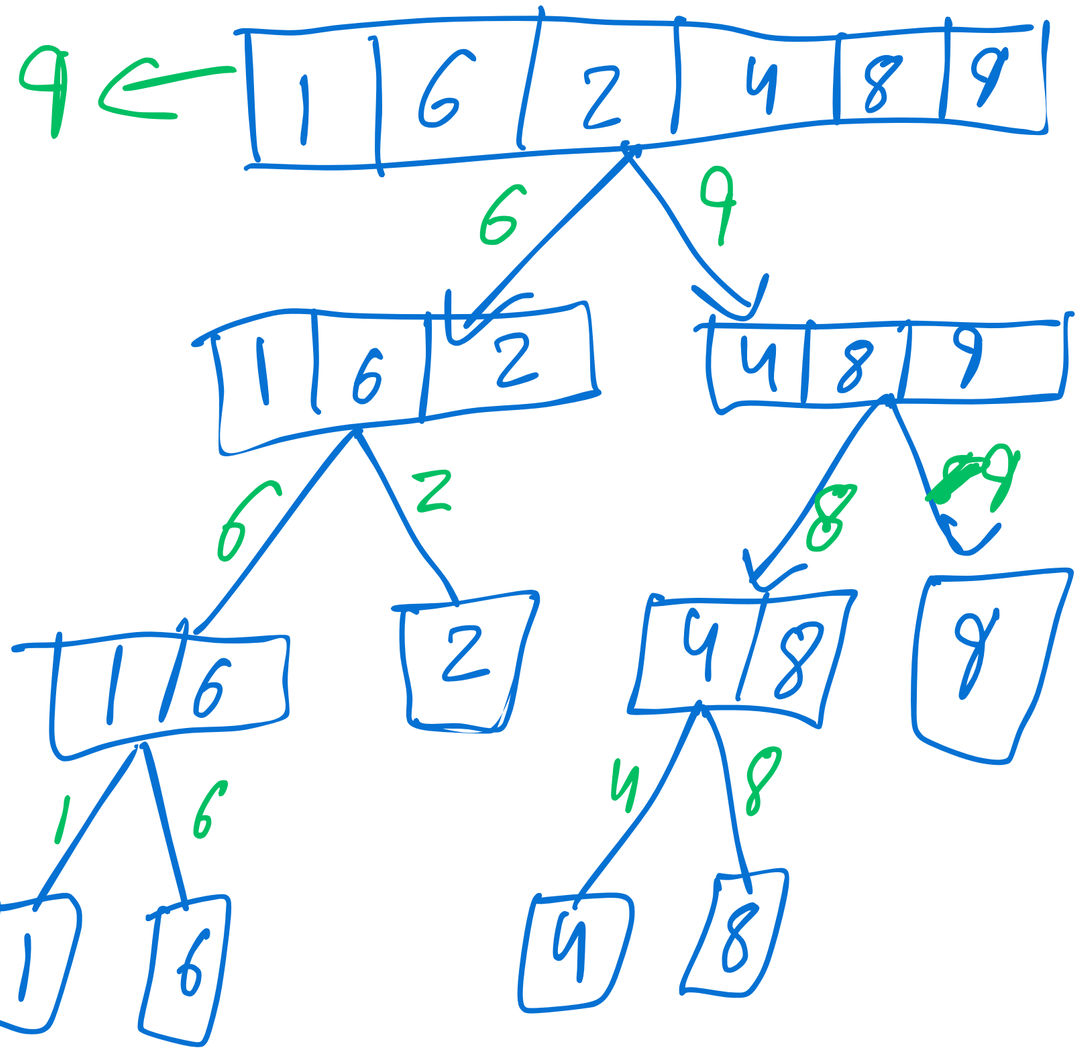
0, 1, 1, 2, 3, 5, 8, 13. . . .

```
9     def mystery(x):
10        if len(x) == 1:
11            return x[0]
12        else:
13            y = mystery(x[1:])
14            if x[0] > y:
15                return x[0]
16            else:
17                return y
18
```

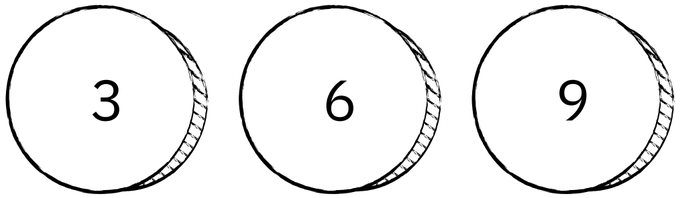


```python
def mistero(x):
    a = len(x)
    if a == 1:
        return x[0]
    else:
        y = mistero(x[:a // 2])
        z = mistero(x[a // 2:])
        if z > y:
            return z
        else:
            return y

```



$23




```
39  def can_make_change(amount, coins):
40      # base case: success
41      if amount == 0
42          return True
43
44      # base case: failure
45      if amount < 0 or len(coins) == 0:
46          return False
47
48      # recursive case: handle two possibilities, either:
49      # 1. another of this coin value gets used, or
50      # 2. we don't need another coin of this value
51      coin = coins[-1]
52      return (can_make_change(amount-coin, coins)
53              or can_make_change(amount, coins[:-1]))
```

Dec	Bin	Oct	Hex
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A

Dec	Bin	Oct	Hex
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14
21	10101	25	15

Power	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Decimal	128	64	32	16	8	4	2	1
Binary	10000000	1000000	100000	10000	1000	100	10	1

Base N to decimal

Sum of $\text{decimalDigit} \times \text{base}^{position}$

$$A5B_{16} = 10 \times 16^2 + 5 \times 16^1 + 11 \times 16^0 = 2651_{10}$$

101011_2

$= 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 \ldots$

$=$

Hex Code	Red	Green	Blue	Guess Colour
#FFFF00	255	FF $15 \times 16^1 + 15 \times 16^0 = 255$	00 $0 \times 16^1 + 0 \times 16^0 = 0$	Yellow
#000000	0	0	0	Black
#008080	0	80 $8 \times 16^1 + 0 \times 16^0 = 128$	80 $8 \times 16^1 + 0 \times 16^0 = 128$	Cyan
#BD00FF	$11 \times 16^1 + 13 \times 16^0 =$ 189	0	255	Purple

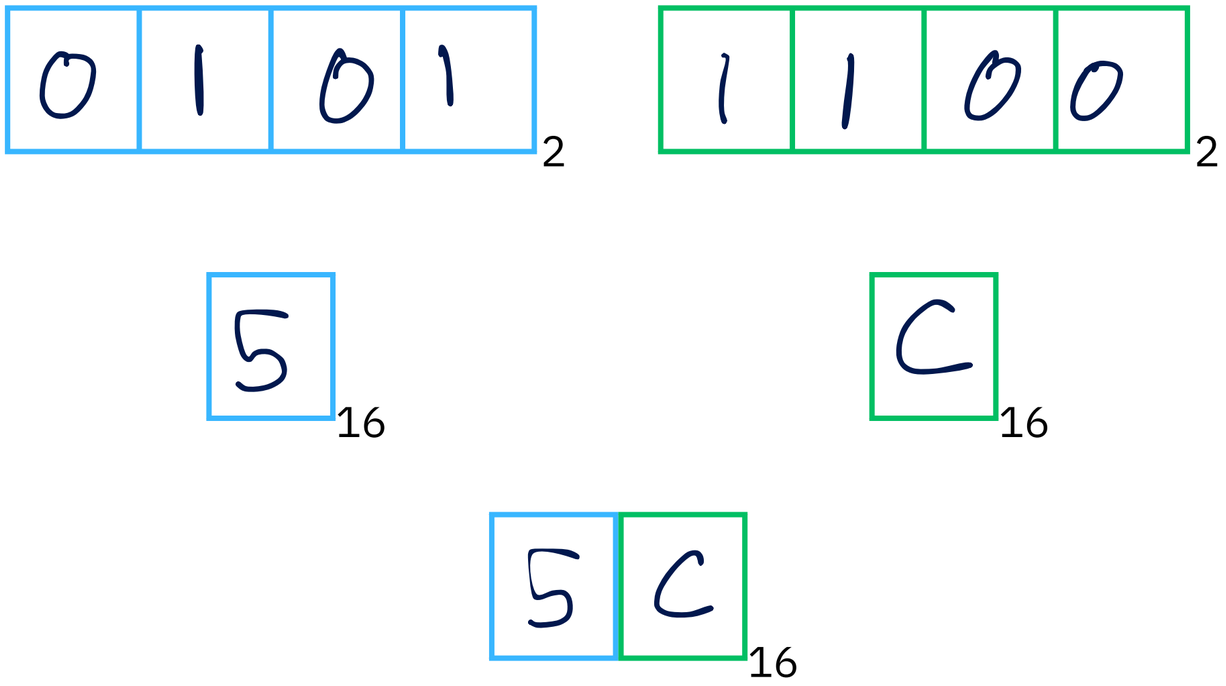
Converting Bases in Python

```
bin(287)
oct(382)
hex(465)
```

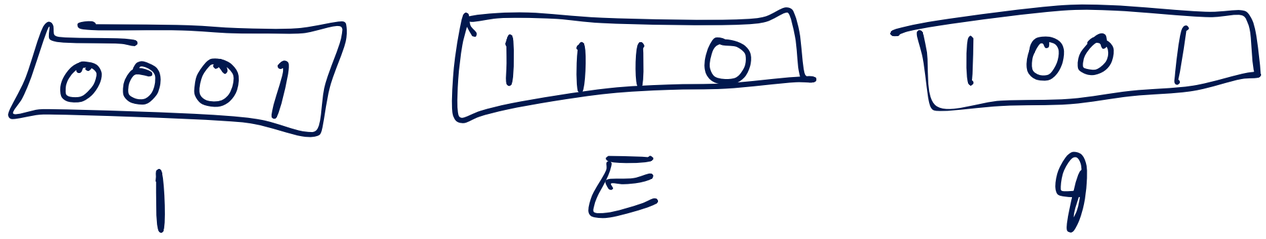
```
467
0b1010110
0o224
0xFFA24
```

Convert the binary number 1011100 to hexadecimal by filling in the boxes in the following diagram:

$$1011100_2 = 56_{16}$$



$$11110100 1_2$$



$$1E9_{16}$$

Write a recursive function to flatten a nested list of integers.

For example,

```
>>> flatten_list([[0, 1], 2, [3, 4, [5, 6, [7], 8]]])
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Write a Python function to convert a hexadecimal colour code into its corresponding RGB integer values. The function should accept a hex code string (e.g. **'#1E90FF'**) and return a tuple containing the red, green, and blue values as integers.

```
>>> hex_to_rgb('#1E90FF')
(30, 144, 255)
```